Court Connect: Technical Document

Oct 15, 2026 · @Yosef

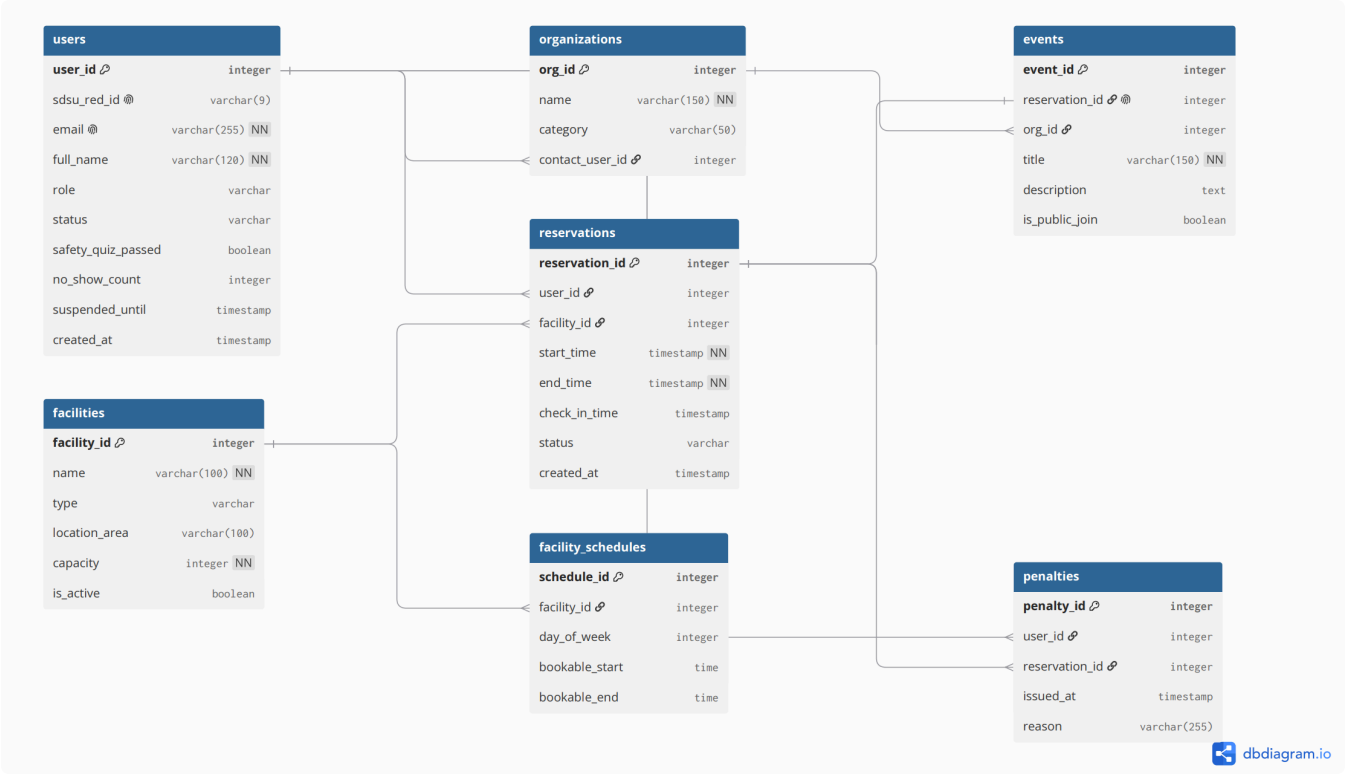
Team	CourtConnect
Members	Yosef Pilip, Riley Kenela, Hudson Mitchell
Course	CS 514, Database Theory and Implementation, San Diego State University
Phase	Draft
Repository	Hmitchell48/CS-514-Project · live site

CourtConnect is one reservation system for every SDSU athletic space: courts, fields, pool lanes, gyms and studios. Students reserve a 2-hour slot and check in on arrival. Clubs book larger spaces and post events. The front-end booking flow is built; the relational database described below is the design it will run on.

1. Design

The database has 7 tables in 3NF: users, facilities, facility_schedules, reservations, organizations, events and penalties. They follow the team's ER diagram below. Types are written for PostgreSQL.

ER diagram



Entities

- **users**: students, club contacts and admins, with their no-show standing.
- **facilities**: one bookable space, such as an ARC court or a pool lane, with its capacity.
- **facility_schedules**: the hours a facility can be booked on each day of the week.
- **reservations**: one user holding one facility from a start time to an end time.
- **organizations**: clubs, fraternities and sororities, each with a contact user.
- **events**: an organization's activity held on one reservation, shown on the events feed.
- **penalties**: one record per no-show or rule violation against a user.

Relationships

Relationship	Cardinality	How it is stored
A user makes many reservations	1 : N	`reservations.user_id`
A facility has many reservations	1 : N	`reservations.facility_id`
A facility has many schedule windows	1 : N	`facility_schedules.facility_id`

Relationship	Cardinality	How it is stored
A user is the contact for many organizations	1 : N	`organizations.contact_user_id`
An organization hosts many events	1 : N	`events.org_id`
A reservation holds at most one event	1 : 0..1	`events.reservation_id` (unique)
A user receives many penalties	1 : N	`penalties.user_id`
A reservation leads to many penalties	1 : N	`penalties.reservation_id`

Table schema

Columns, types and keys are from the ER diagram. The CHECK and DEFAULT constraints are added in the DDL. PK = primary key, FK = foreign key.

users

Column	Type	Key	Constraints
user_id	SERIAL	PK	
sdsu_red_id	VARCHAR(9)		UNIQUE, CHECK 9 digits
email	VARCHAR(255)		NOT NULL, UNIQUE
full_name	VARCHAR(120)		NOT NULL
role	VARCHAR(10)		CHECK IN ('student', 'club', 'admin'), DEFAULT 'student'
status	VARCHAR(10)		CHECK IN ('active', 'warned', 'banned'), DEFAULT 'active'
safety_quiz_passed	BOOLEAN		NOT NULL, DEFAULT FALSE
no_show_count	INTEGER		NOT NULL, DEFAULT 0, CHECK >= 0
suspended_until	TIMESTAMP		NULL unless temporarily locked out
created_at	TIMESTAMP		NOT NULL, DEFAULT now()

Open question: the diagram has no password column. Sign-up needs one, or SDSU single sign-on.

facilities

Column	Type	Key	Constraints
facility_id	SERIAL	PK	
name	VARCHAR(100)		NOT NULL
type	VARCHAR(20)		CHECK IN ('court', 'field', 'pool', 'gym', 'studio', 'other')
location_area	VARCHAR(100)		e.g. 'Aztec Recreation Center'
capacity	INTEGER		NOT NULL, CHECK > 0 (students who can hold the same time)
is_active	BOOLEAN		NOT NULL, DEFAULT TRUE

facility_schedules

Column	Type	Key	Constraints
schedule_id	SERIAL	PK	
facility_id	INTEGER	FK → facilities	NOT NULL, ON DELETE CASCADE
day_of_week	INTEGER		NOT NULL, CHECK BETWEEN 0 AND 6 (0 = Sunday)
bookable_start	TIME		NOT NULL
bookable_end	TIME		NOT NULL, CHECK bookable_end > bookable_start

reservations

Column	Type	Key	Constraints
reservation_id	SERIAL	PK	
user_id	INTEGER	FK → users	NOT NULL

Column	Type	Key	Constraints
facility_id	INTEGER	FK → facilities	NOT NULL
start_time	TIMESTAMP		NOT NULL
end_time	TIMESTAMP		NOT NULL, CHECK end_time > start_time
check_in_time	TIMESTAMP		NULL until check-in
status	VARCHAR(12)		CHECK IN ('booked', 'checked-in', 'no-show', 'cancelled'), DEFAULT 'booked'
created_at	TIMESTAMP		NOT NULL, DEFAULT now()

organizations

Column	Type	Key	Constraints
org_id	SERIAL	PK	
name	VARCHAR(150)		NOT NULL, UNIQUE
category	VARCHAR(50)		e.g. 'club', 'fraternity', 'sorority'
contact_user_id	INTEGER	FK → users	NOT NULL

events

Column	Type	Key	Constraints
event_id	SERIAL	PK	
reservation_id	INTEGER	FK → reservations	NOT NULL, UNIQUE
org_id	INTEGER	FK → organizations	NOT NULL
title	VARCHAR(150)		NOT NULL
description	TEXT		

Column	Type	Key	Constraints
is_public_join	BOOLEAN		NOT NULL, DEFAULT FALSE (any student may join, e.g. a pickup session)

penalties

Column	Type	Key	Constraints
penalty_id	SERIAL	PK	
user_id	INTEGER	FK → users	NOT NULL
reservation_id	INTEGER	FK → reservations	NULL for penalties not tied to a booking
issued_at	TIMESTAMP		NOT NULL, DEFAULT now()
reason	VARCHAR(255)		NOT NULL, e.g. 'no-show'

Normalization

Every table is in third normal form: each non-key column depends on the whole primary key and on nothing else.

- **facility_schedules split out of facilities.** A facility's hours are a repeating group: at least 7 open/close pairs, more when a day has split hours. Kept in facilities they would need 14+ columns or a list in one cell, which breaks 1NF. One row per window fixes that.
- **events point at a reservation instead of copying its time and place.** If events stored facility_id, start and end, those would repeat the reservation that holds the space, and the two could disagree. With a unique reservation_id, event_id → reservation_id → facility and time, so each fact is stored once, in reservations.
- **penalties split out of users.** A user's penalty history is another repeating group, so each penalty is its own row. users keeps only `no_show_count` and `suspended_until`, a summary for fast checks at booking time. That summary is derived from penalties, so a trigger (section 3) updates both together and they can't drift.

2. Implementation

The front end and booking flow are built and live on GitHub Pages; the database and API are the next milestone.

Tech stack

Layer	Choice	Status
Frontend	Plain HTML, CSS and JavaScript, no framework or build step	Built (`index.html`, `js/booking.js`, `css/styles.css`)
Hosting (frontend)	GitHub Pages	Live
Data layer today	Browser `localStorage` plus generated sample data	Built, temporary
Database	PostgreSQL	Planned
Backend	REST API between the page and the database	Planned

Open question: the team has not yet picked the backend framework or database host.

How the pieces connect

The booking page never touches storage directly. Every read and write goes through one `api` object in `js/booking.js` with seven methods: `signUp`, `logIn`, `getDay`, `listReservations`, `reserve`, `cancel` and `checkIn`. Each returns a promise, so connecting the database means replacing those seven bodies with `fetch()` calls to the backend, which runs the SQL in section 3 against PostgreSQL. Today those methods read and write `localStorage`, and other students' bookings come from a seeded hash so the schedule looks realistic and stays stable between reloads.

Key features built

- **Sign-up wall.** Validates a 9-digit RedID, an `@sdsu.edu` email and an 8+ character password. Two test accounts log in until the Users table exists.

- **Availability view.** Pick one of 8 venues and one of the next 7 days. Each space shows its seven 2-hour slots (7 am to 9 pm) as Open, N spots left, Booked, Full, Walk-up, Event, Closed or Ended.
- **Real facility data.** 74 spaces across 8 venues, with fall 2026 hours from arc.sdsu.edu, including per-space hours for the climbing walls.
- **Reservations.** Confirm dialog, then booking. Enforces one reservation per day, capacity per slot, walk-up windows, venue hours and no booking a slot that has started.
- **Check-in and no-shows.** Check-in opens 15 minutes before a slot and closes at its end. A booking that ends without check-in shows as a no-show.
- **Cancel.** Allowed until the slot starts; the slot frees up for others.
- **Club events.** Sample events (Spikeball, Volleyball, Swim and Soccer clubs) block their slots and appear under the space name.

Not built yet: club event posting, open sessions students can join, penalties and the admin view. The events and penalties tables in the design cover them.

3. Course Concepts Applied

Each rule the front end checks today in `api.reserve` moves into the database as a key, constraint, trigger or transaction, so no client can bypass it.

Primary and foreign keys

Every table has a SERIAL surrogate primary key; `sdsu_red_id` and `email` stay UNIQUE as natural candidate keys. Foreign keys tie every reservation to a real user and facility, every event to an organization and a reservation, and every penalty to a user.

```
CREATE TABLE reservations (
  reservation_id SERIAL PRIMARY KEY,
  user_id        INTEGER     NOT NULL REFERENCES users(user_id),
  facility_id    INTEGER     NOT NULL REFERENCES facilities(facility_id),
  start_time     TIMESTAMP   NOT NULL,
  end_time       TIMESTAMP   NOT NULL,
  check_in_time  TIMESTAMP,
  status         VARCHAR(12) NOT NULL DEFAULT 'booked'
                 CHECK (status IN ('booked', 'checked-in', 'no-show',
'cancelled')),
  created_at     TIMESTAMP   NOT NULL DEFAULT now(),
  CHECK (end_time > start_time)
);
```

SQL queries (SELECT, INSERT, UPDATE, DELETE)

```
-- SELECT: a student's upcoming reservations (backs listReservations)
SELECT r.reservation_id, f.name, f.location_area, r.start_time, r.end_time,
r.status
FROM reservations r
JOIN facilities f ON f.facility_id = r.facility_id
WHERE r.user_id = 1 AND r.start_time >= now() AND r.status <> 'cancelled'
ORDER BY r.start_time;

-- INSERT: book a slot (backs reserve)
INSERT INTO reservations (user_id, facility_id, start_time, end_time)
VALUES (1, 3, '2026-10-20 17:00', '2026-10-20 19:00');

-- UPDATE: check in, allowed from 15 minutes before the start (backs checkIn)
UPDATE reservations
SET status = 'checked-in', check_in_time = now()
WHERE reservation_id = 42 AND user_id = 1 AND status = 'booked'
  AND now() BETWEEN start_time - INTERVAL '15 minutes' AND end_time;

-- DELETE: an admin removes a schedule window
DELETE FROM facility_schedules WHERE schedule_id = 12;
```

Cancelling is an UPDATE to `status = 'cancelled'`, not a DELETE, so booking and no-show history is kept.

Joins, aggregates, subqueries

```
-- Join + aggregate: reservations per facility over the last 30 days
SELECT f.location_area, f.name, COUNT(r.reservation_id) AS bookings
FROM facilities f
LEFT JOIN reservations r
  ON r.facility_id = f.facility_id
 AND r.status <> 'cancelled'
 AND r.start_time >= now() - INTERVAL '30 days'
GROUP BY f.facility_id, f.location_area, f.name
ORDER BY bookings DESC;

-- Subquery: students with more no-shows than average
SELECT full_name, no_show_count
FROM users
WHERE no_show_count > (SELECT AVG(no_show_count) FROM users);

-- Four-table join: open sessions any student can join this week
SELECT e.title, o.name AS organization, f.name AS facility, r.start_time
FROM events e
JOIN reservations r  ON r.reservation_id = e.reservation_id
JOIN organizations o ON o.org_id = e.org_id
JOIN facilities f    ON f.facility_id = r.facility_id
WHERE e.is_public_join
  AND r.start_time BETWEEN now() AND now() + INTERVAL '7 days';
```

Constraints

Rule	How the database enforces it
One reservation per student per day	`CREATE UNIQUE INDEX one_per_day ON reservations (user_id, (start_time::date)) WHERE status <> 'cancelled';`
No double-booking past a facility's capacity	Trigger `check_booking` (below) counts active reservations that overlap the new one against `facilities.capacity`
Bookings only inside open hours	`check_booking` also requires a `facility_schedules` window for that weekday covering the start and end

Rule	How the database enforces it
Times make sense	`CHECK (end_time > start_time)` on reservations, `CHECK (bookable_end > bookable_start)` on facility_schedules
One event per reservation	`UNIQUE (reservation_id)` on events
Valid statuses and roles	`CHECK (... IN (...))` on every status and role column
One account per student	`UNIQUE (sdsu_red_id)` and `UNIQUE (email)` on users

Views

```
-- Facilities open today, with how many bookings each window has
CREATE VIEW todays_open_facilities AS
SELECT f.facility_id, f.name, f.location_area, f.capacity,
       s.bookable_start, s.bookable_end,
       COUNT(r.reservation_id) AS booked_today
FROM facilities f
JOIN facility_schedules s
  ON s.facility_id = f.facility_id
 AND s.day_of_week = EXTRACT(DOW FROM CURRENT_DATE)
LEFT JOIN reservations r
  ON r.facility_id = f.facility_id
 AND r.status <> 'cancelled'
 AND r.start_time >= CURRENT_DATE + s.bookable_start
 AND r.start_time <  CURRENT_DATE + s.bookable_end
WHERE f.is_active
GROUP BY f.facility_id, f.name, f.location_area, f.capacity,
         s.schedule_id, s.bookable_start, s.bookable_end;
```

A second view, `events_feed`, joins events, reservations, organizations and facilities for upcoming, non-cancelled events; it backs the public events feed.

Stored procedures and triggers

- **`check_booking` (BEFORE INSERT trigger on reservations).** Rejects the insert if the user is banned, suspended (`suspended_until > now()`) or hasn't passed the safety quiz; if no schedule window covers the time; or if overlapping active reservations

already fill the facility's capacity.

- **`record_no_show` (AFTER UPDATE trigger on reservations).** When status becomes 'no-show', inserts a penalties row and updates the user's `no_show_count`, `status` and `suspended_until` in the same statement, so the summary in users always matches penalties. One no-show warns and suspends; three ban.
- **`mark_no_shows()` (stored procedure, run every 15 minutes).** Sets every 'booked' reservation whose `end_time` has passed to 'no-show', which fires `record_no_show`.

```
CREATE FUNCTION record_no_show() RETURNS trigger AS $$
BEGIN
  IF NEW.status = 'no-show' AND OLD.status <> 'no-show' THEN
    INSERT INTO penalties (user_id, reservation_id, reason)
    VALUES (NEW.user_id, NEW.reservation_id, 'no-show');

    UPDATE users
    SET no_show_count   = no_show_count + 1,
        status          = CASE WHEN no_show_count + 1 >= 3 THEN 'banned' ELSE
'warned' END,
        suspended_until = CASE WHEN no_show_count + 1 < 3 THEN now() +
INTERVAL '7 days' END
    WHERE user_id = NEW.user_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER no_show_penalty
AFTER UPDATE OF status ON reservations
FOR EACH ROW EXECUTE FUNCTION record_no_show();
```

The 7-day suspension is a placeholder; the site still lists the lockout length as a TODO.

Transactions

A club booking writes two rows: the reservation that holds the space and the event that goes on the feed. Both commit or neither does, so there is never an event without a space or a held space without its event. The `FOR UPDATE` lock on the facility stops two bookings racing for the last spot.

```
BEGIN;
  SELECT capacity FROM facilities WHERE facility_id = 7 FOR UPDATE;
  INSERT INTO reservations (user_id, facility_id, start_time, end_time)
  VALUES (5, 7, '2026-10-21 13:00', '2026-10-21 15:00')
  RETURNING reservation_id;              -- say it returns 88
  INSERT INTO events (reservation_id, org_id, title, is_public_join)
  VALUES (88, 2, 'Spikeball Club open session', TRUE);
COMMIT;   -- any error (full, outside hours, missing title) -> ROLLBACK
```

4. Screenshots

Four of the six screens exist on the live site today; log in with `dev@sdsu.edu` / `courtconnect-dev` to reach them. Insert one image above each caption.

#	Screen	Caption	Status
1	Home	The CourtConnect home page and sign-up wall: students register with their RedID and @sdsu.edu email.	Ready to capture
2	Facility availability	The availability grid for the Aztec Recreation Center: each space's 2-hour slots show open, booked, full, walk-up or event.	Ready to capture
3	Reservation flow	The confirm dialog before booking, with the space, time, venue and the one-per-day reminder.	Ready to capture
4	Your reservations	A student's bookings with Check in and Cancel buttons and booked, checked-in or no-show badges.	Ready to capture
5	Open sessions	Club sessions marked open for any student to join, on the events feed.	Not built yet
6	Club events / admin view	A club booking a space and posting its event; the admin view of facilities and penalties.	Not built yet

5. User Guide

Steps marked *(planned)* describe features whose tables exist in the design but whose screens are not built yet.

Student

1. **Sign up.** Enter your name, 9-digit RedID, @sdsu.edu email and a password of 8+ characters.
2. **Pass the facility quiz** *(planned)*. Complete the safety and etiquette quiz. You can't reserve until you pass.
3. **Find a court.** Pick a venue from the dropdown, such as Aztec Recreation Center or Aztec Tennis Center, then a day in the next 7.
4. **Check availability.** Each space shows its slots from 7 am to 9 pm. *Open* or *N spots* can be booked; *Walk-up* (11 am and 1 pm) is first come, first served; *Event* is held by a club.
5. **Reserve it.** Click an open slot and confirm. You get one reservation per day.
6. **Check in.** In *Your reservations*, press **Check in** from 15 minutes before your slot until it ends. Miss it and it counts as a no-show: one brings a temporary suspension, three a permanent ban.
7. **Cancel** any time before the slot starts to free it for someone else.
8. **Join an open session** *(planned)*. On the events feed, sessions marked open to join, such as a Spikeball Club pickup session, are open to any student. Show up at the listed time.

Club or organization

1. **Get set up** *(planned)*. An admin adds the organization with its name, category and contact user.
2. **Book the space and post the event** *(planned)*. The contact picks a facility, such as ENS 700 Field or ARC North Court, and a start and end time, then adds a title, a description and whether any student can join. The reservation and the event are saved together.
3. **Show up on the feed.** The event appears on the events feed and blocks that time for other students.

Admin

1. **Manage facilities** *(planned)*. Add or edit facilities and their capacity, set the bookable windows for each weekday, or mark a facility inactive to take it off the schedule.

2. **Manage organizations** *(planned)*. Add clubs and set or change their contact user.
3. **Handle penalties** *(planned)*. Review penalties and users who are warned or banned, and lift a suspension when appropriate.

The current schema has no approval step for club events. If admins should approve them, events needs a status column.

6. AI Usage Disclosure

Where AI was used

- **This document.** Claude (Anthropic) drafted it from our repository and our ER diagram. It wrote the schema tables from the diagram, and proposed the CHECK constraints, SQL examples, views, triggers, transaction and normalization write-up.
- **Website.** Claude also made site edits: adding team photos and roles, removing the fee and payment content (including the payments table in the ER diagram image), making every page use the same schema, and publishing this PDF.
- **Code.** *[Team to fill in: e.g. brainstorming the feature list, starter code for the booking page, debugging, CSS.]*

What our team changed or extended

[Team to fill in with specific examples. Suggestions to check before submitting:]

- [ ] Confirm PostgreSQL and the backend choice, or replace them with what we actually use.
- [ ] Decide how passwords are stored; the ER diagram has no column for them yet.
- [ ] Run every SQL example against our real DDL and fix anything that fails.
- [ ] Replace the placeholder 7-day suspension with our real number.
- [ ] Describe what each of us wrote or rewrote by hand.